

AI Fitness Coach Platform

Backend Foundation and Infrastructure Documentation

Production-ready backend architecture for an AI-driven, personalized fitness coaching platform — designed from inception to support retrieval-augmented generation, vector similarity search, and long-term user memory.

Table of Contents

1	Executive Summary	4
2	System Overview	4
3	Technical Stack	5
4	System Architecture	6
5	Project Structure	6
6	Containerized Infrastructure	8
6.1	Docker Compose Architecture	8
6.2	PostgreSQL with pgvector	8
6.3	Backend Container	9
7	Database Design	10
7.1	User Entity	10
7.2	User Profile Entity	10
7.3	UUID Primary Keys	10
8	Database Migration Strategy	11
9	User Registration Workflow	12
9.1	Security Considerations	12
10	API Documentation	12
11	Development Environment & Workflow	14
12	Achievements of Phase One	14
13	Future Expansion	15
14	Key Engineering Decisions	16
15	Scalability Considerations	16

01 — OVERVIEW

1 Executive Summary

This document describes the design and implementation of the foundational backend infrastructure for the AI Fitness Coach platform.

The objective of this phase was to establish a production-ready environment capable of supporting future AI-driven capabilities, including personalized workout generation, retrieval-augmented generation (RAG), vector similarity search, and long-term user memory. Rather than treating these capabilities as a later add-on, the backend was architected from the outset to accommodate them without structural rework.

The resulting architecture delivers:

- A containerized, reproducible development environment
- A scalable REST API built on FastAPI
- A PostgreSQL database extended with vector search capabilities via pgvector
- Version-controlled schema evolution through Alembic migrations
- Secure user registration and profile management
- A modular project structure designed for AI feature integration

All components run inside Docker containers, ensuring identical execution environments across development machines, CI pipelines, and deployment targets.

Scope note. This phase deliberately establishes the platform foundation before introducing AI-specific functionality. Every infrastructure decision documented here was made with the subsequent AI layer in mind.

02 — FOUNDATIONS

2 System Overview

The current implementation establishes the core backend services required to operate the platform in production, while intentionally deferring AI-specific functionality to subsequent phases.

Infrastructure Components

FastAPI application server, PostgreSQL 16, pgvector extension, Alembic migrations, Docker Compose orchestration, SQLAlchemy ORM, and a Pydantic validation layer.

Implemented Functionality

User account creation, user profile management, schema versioning, secure password storage, containerized deployment, and a health monitoring endpoint.

This separation of concerns — infrastructure first, intelligence second — is a deliberate engineering trade-off: it front-loads the operational risk (deployment, data modeling, security) so that later AI features can be built on a stable, already-verified foundation.

03 — STACK

3 Technical Stack

Each component of the stack was selected for production maturity, ecosystem support, and direct alignment with the platform's AI roadmap.

Layer	Technology
API Framework	FastAPI
Database	PostgreSQL 16
Vector Database Extension	pgvector
ORM	SQLAlchemy
Validation	Pydantic
Migration System	Alembic
Containerization	Docker
Container Orchestration	Docker Compose
Password Hashing	bcrypt

04 — ARCHITECTURE

4 System Architecture

At the current phase, the platform follows a straightforward request path: client traffic reaches the FastAPI backend, which persists and retrieves data from PostgreSQL. The pgvector extension is provisioned inside the same database instance, ready to serve embedding storage and similarity search once AI features are activated. Both the API and database run as isolated Docker services communicating over Docker's internal network.

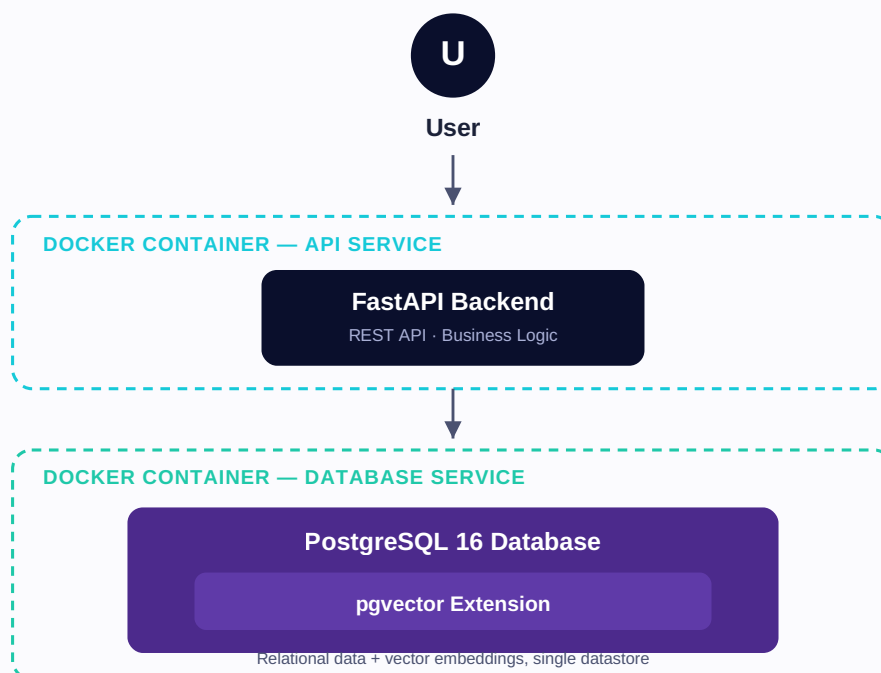


Fig. 1 — **Request path and container boundaries.** The FastAPI service and PostgreSQL/pgvector service run as independent Docker containers connected over an internal network.

This topology keeps the request path short and observable today, while the presence of pgvector inside the database container means that RAG and similarity-search workloads can be added as new routers and services without introducing a new datastore.

05 — CODEBASE

5 Project Structure

The codebase follows a modular layout that separates infrastructure configuration, business logic, database models, and API routes. This separation reduces coupling between layers and makes the

addition of future AI modules (embedding services, RAG routers, coaching logic) a matter of adding new packages rather than restructuring existing ones.

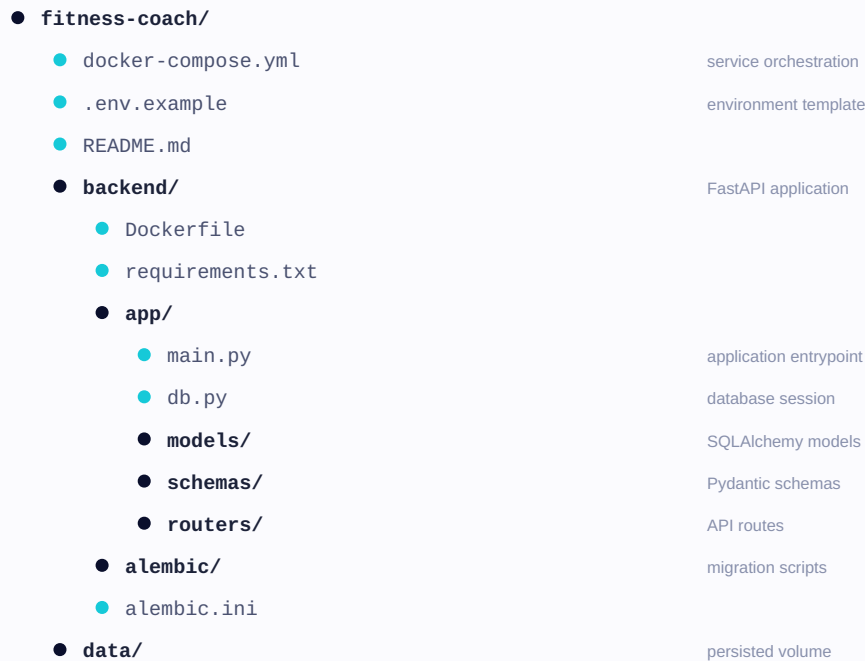


Fig. 2 — **Project directory structure.** Backend logic, migrations, and infrastructure configuration are cleanly separated.

The `app/models`, `app/schemas`, and `app/routers` packages map directly to SQLAlchemy models, Pydantic schemas, and FastAPI routers respectively — a convention that keeps the data layer, validation layer, and transport layer independently testable.

06 — INFRASTRUCTURE

6 Containerized Infrastructure

The entire platform runs inside Docker containers to guarantee environment consistency and eliminate dependency conflicts between developer machines, CI runners, and production hosts. This eliminates an entire category of "works on my machine" defects and reduces onboarding time for new engineers to a single `docker compose up`.

Benefits realized

Reproducible builds, simplified onboarding, dependency isolation, platform independence, and a direct path to production deployment.

Developer prerequisites

Docker Desktop or Docker Engine, Git, and a code editor. No local installation of Python or PostgreSQL is required.

6.1 Docker Compose Architecture

The platform currently consists of two independently deployable services:

- **API service** — handles HTTP requests, authentication logic, data validation, and business logic execution.
- **Database service** — provides persistent storage for user data and, going forward, vector embeddings.

The two services communicate exclusively through Docker's internal networking layer, keeping the database unreachable from outside the container network by default.

6.2 PostgreSQL with pgvector

PostgreSQL was selected as the primary datastore for its reliability, transactional guarantees, and mature ecosystem. The `pgvector` extension was enabled from day one to support upcoming AI capabilities:

- Semantic search over exercises and content
- Retrieval-Augmented Generation (RAG) pipelines
- Similarity search across user history
- Long-term user memory retrieval
- General-purpose embedding storage

Vector functionality is not yet exercised by the application logic in this phase, but provisioning it now avoids a disruptive data-store migration later — a materially cheaper decision than retrofitting vector search onto a relational schema after production data exists.

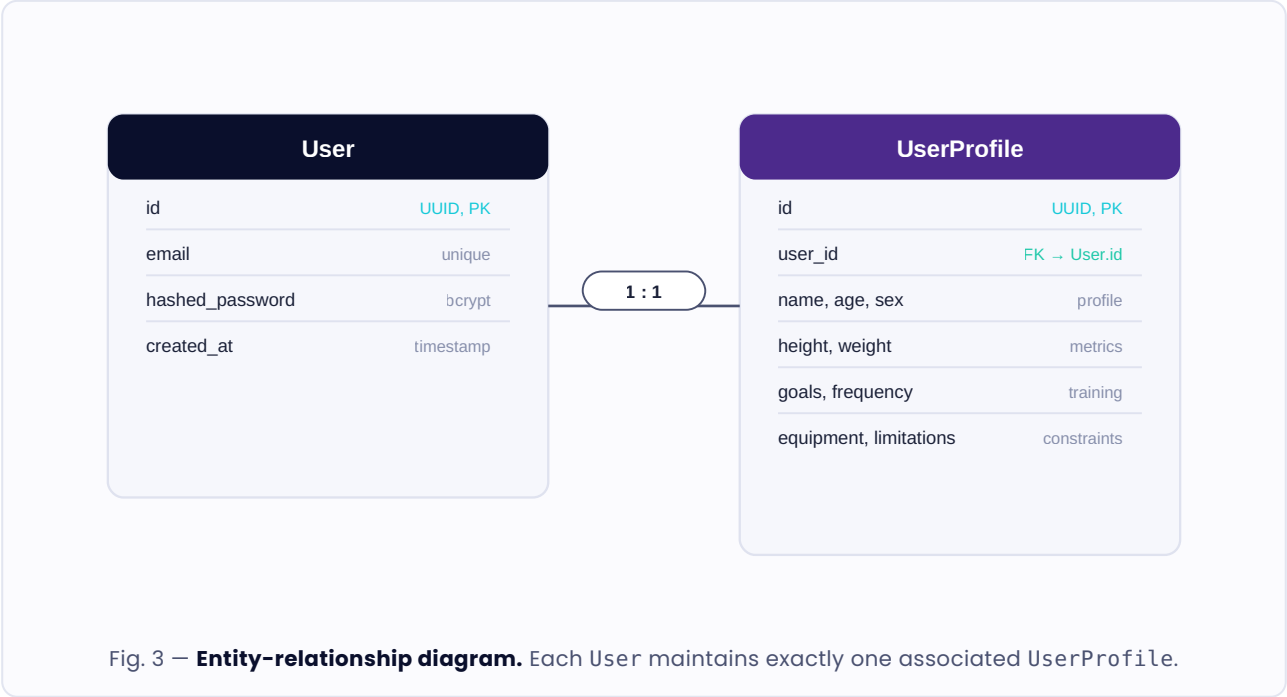
6.3 Backend Container

The FastAPI application runs inside an isolated Python container responsible for installing dependencies, executing the application server, exposing API endpoints, and managing hot reload during development — guaranteeing consistent behavior across local development, testing, and production.

07 — DATA MODEL

7 Database Design

The schema currently models two entities, deliberately separating authentication data from fitness-domain data.



7.1 User Entity

The `User` entity stores authentication-related information only: a unique identifier, email address, and hashed password. Isolating authentication data from fitness data improves security posture and maintains a clean separation of concerns between "who the user is" and "what the user is doing."

7.2 User Profile Entity

The `UserProfile` entity stores fitness-related attributes: name, age, biological sex, height, weight, training experience, primary fitness goals, training frequency, available equipment, and physical limitations. Each profile maintains a one-to-one relationship with a user account.

7.3 UUID Primary Keys

The system uses UUID identifiers rather than sequential integers across all entities. This choice yields improved security through non-predictability, better support for distributed systems, and easier data synchronization across future services — a decision that pays off disproportionately once the platform moves toward a multi-service architecture.

08 — SCHEMA EVOLUTION

8 Database Migration Strategy

Schema evolution is managed through Alembic migrations. Every database modification is represented as a version-controlled migration file, allowing the project to:

- Track complete schema history
- Reproduce any historical database state
- Deploy schema changes safely and predictably
- Collaborate across multiple developers without conflicting schema drift

This approach significantly reduces deployment risk and ensures consistency between development, staging, and production environments.

09 — CORE FLOW

9 User Registration Workflow

The registration endpoint is the first business feature implemented on top of the infrastructure layer, and it exercises the full stack: validation, persistence, and transactional integrity.

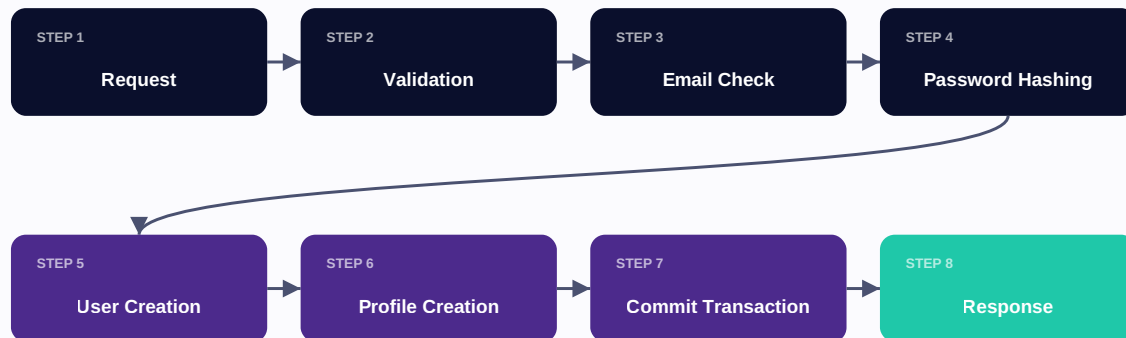


Fig. 4 — **Registration request lifecycle**, from incoming request to sanitized response.

9.1 Security Considerations

Password Hashing

Passwords are never stored in plaintext. bcrypt hashing transforms passwords before database storage, so a database compromise does not expose recoverable credentials.

Input Validation

Pydantic schemas validate email format, positive numeric values, required fields, and data types at the API boundary. Invalid requests are rejected before reaching business logic, keeping malformed data out of the persistence layer entirely.

Transaction Safety

The registration operation executes inside a single database transaction spanning both user and profile creation. If any step fails, all changes are rolled back automatically, preserving database consistency and preventing orphaned records.

10 — DEVELOPER EXPERIENCE

10 API Documentation

FastAPI automatically generates interactive API documentation from OpenAPI specifications, giving developers direct access to a Swagger UI, request and response schemas, and built-in endpoint testing tools. This eliminates a class of documentation drift common in hand-maintained API docs and materially improves integration speed for any future frontend or partner client.

11 — OPERATIONS

11 Development Environment & Workflow

The development environment is fully containerized, so a new contributor can go from a fresh checkout to a running stack without installing Python or PostgreSQL locally.

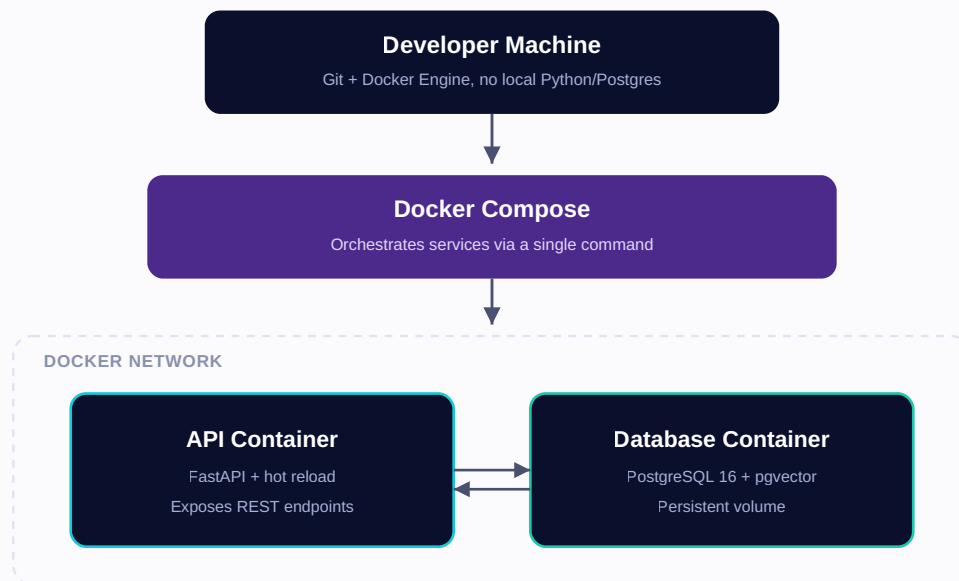


Fig. 5 — **Local development topology.** Docker Compose orchestrates the API and database containers on the developer's machine.

The project follows a standard Git-based workflow: pull latest changes, implement features, commit, and push to GitHub. As the team scales, a feature-branching strategy can be introduced to further improve release stability without disrupting the current setup.

12 — STATUS

12 Achievements of Phase One

At the conclusion of this implementation phase, the platform provides:

- A fully reproducible development environment
- Containerized infrastructure across all services
- A scalable backend architecture
- Secure user authentication foundations
- Persistent user profile storage

- Version-controlled database schema management
- Future-ready vector database support

These components establish the technological foundation required for the AI capabilities introduced in subsequent development phases.

13 — ROADMAP

13 Future Expansion

This phase intentionally focuses only on the features described above. Subsequent development phases are expected to introduce:

- Workout session tracking
- Exercise history storage
- Personalized recommendations
- Long-term user memory
- Retrieval-Augmented Generation pipelines
- Embedding generation and retrieval
- AI coaching capabilities

The architectural decisions made in this phase — particularly the early adoption of pgvector, UUID identifiers, and a modular router structure — were made specifically so that these extensions can be added without major refactoring.

14 — RATIONALE

14 Key Engineering Decisions

The technology choices in this phase were not arbitrary. Each carries a specific trade-off that was evaluated against the platform's near-term and long-term requirements.

Why FastAPI

FastAPI provides native async support, automatic OpenAPI documentation, and first-class Pydantic integration — reducing boilerplate for validation while keeping request throughput high, which matters once AI inference calls introduce latency into the request path.

Why PostgreSQL

PostgreSQL offers strong transactional guarantees, a mature extension ecosystem, and proven reliability at scale. It is also the natural host for pgvector, avoiding the operational overhead of running a separate vector database alongside the relational store.

Why pgvector was added early

Introducing vector capability at the schema level from day one avoids a future migration that would otherwise require re-provisioning infrastructure, backfilling embeddings, and coordinating downtime once production data exists. The cost of adding it early is near zero; the cost of adding it late is not.

Why Docker

Containerization removes environment drift between development, CI, and production, and reduces onboarding for new engineers to a single command. It also gives the platform a direct, unmodified deployment path to any container-based hosting target.

Why UUIDs

UUID primary keys avoid the predictability and enumeration risks of sequential integers, and they simplify identifier generation across distributed services — a property that becomes valuable the moment the platform splits into multiple deployable services.

15 — FORWARD COMPATIBILITY

15 Scalability Considerations

The foundation was deliberately shaped to absorb the platform's next layer of complexity without structural rework.

RAG systems

pgvector is already provisioned inside the primary database, so retrieval pipelines can query embeddings alongside relational data in a single transaction.

Embeddings storage

Vector columns can be added to existing or new tables through the established Alembic migration path, with no change to the surrounding infrastructure.

Personalized AI coaching

The modular router structure allows coaching logic to be introduced as new FastAPI routers operating on the existing user and profile data.

Long-term memory

UUID-keyed entities and a normalized schema provide a stable foundation for append-only interaction logs and memory retrieval.

Future microservices

Docker Compose orchestration and UUID identifiers already reflect distributed-system assumptions, easing an eventual split into independently deployable services.

Operational continuity

Because every schema change is a tracked migration, scaling the data model forward remains a reviewable, reversible process rather than an ad-hoc one.

16 — SUMMARY

16 Conclusion

The backend foundation of the AI Fitness Coach platform has been established using production-grade infrastructure and deliberate architectural trade-offs. By prioritizing modularity, reproducibility, and security from the outset — and by provisioning vector search capability before it was strictly needed — the platform is positioned to evolve into an AI-driven coaching product without revisiting its core infrastructure.

Foundation readiness

Authentication, schema versioning, containerized deployment, and vector-ready storage are complete. The next development phase can build workout tracking, RAG pipelines, and personalized coaching logic directly on top of this foundation.